

# Untitled16

March 17, 2026

```
[1]: import pandas as pd
from IPython.display import display

df = pd.read_csv("online_retail_II.csv", encoding="latin1")

print("Dataset loaded successfully.")

print("\nFull dataset shape (rows, columns):")
print(df.shape)

print("\nColumn names:")
print(list(df.columns))

print("\nFirst 5 rows:")
display(df.head())

print("\nLast 5 rows:")
display(df.tail())

print("\nData types and non-null counts:")
df.info()

print("\nBasic descriptive statistics:")
display(df.describe(include="all"))

print("\nMissing values per column:")
display(df.isna().sum())
```

Dataset loaded successfully.

Full dataset shape (rows, columns):  
(525461, 8)

Column names:  
['Invoice', 'StockCode', 'Description', 'Quantity', 'InvoiceDate', 'Price',  
'Customer ID', 'Country']

First 5 rows:

|   | Invoice | StockCode | Description                         | Quantity | \ |
|---|---------|-----------|-------------------------------------|----------|---|
| 0 | 489434  | 85048     | 15CM CHRISTMAS GLASS BALL 20 LIGHTS | 12       |   |
| 1 | 489434  | 79323P    | PINK CHERRY LIGHTS                  | 12       |   |
| 2 | 489434  | 79323W    | WHITE CHERRY LIGHTS                 | 12       |   |
| 3 | 489434  | 22041     | RECORD FRAME 7" SINGLE SIZE         | 48       |   |
| 4 | 489434  | 21232     | STRAWBERRY CERAMIC TRINKET BOX      | 24       |   |

|   | InvoiceDate      | Price | Customer ID | Country        |
|---|------------------|-------|-------------|----------------|
| 0 | 01/12/2009 07:45 | 6.95  | 13085.0     | United Kingdom |
| 1 | 01/12/2009 07:45 | 6.75  | 13085.0     | United Kingdom |
| 2 | 01/12/2009 07:45 | 6.75  | 13085.0     | United Kingdom |
| 3 | 01/12/2009 07:45 | 2.10  | 13085.0     | United Kingdom |
| 4 | 01/12/2009 07:45 | 1.25  | 13085.0     | United Kingdom |

Last 5 rows:

|        | Invoice | StockCode | Description                        | Quantity | \ |
|--------|---------|-----------|------------------------------------|----------|---|
| 525456 | 538171  | 22271     | FELTCRAFT DOLL ROSIE               | 2        |   |
| 525457 | 538171  | 22750     | FELTCRAFT PRINCESS LOLA DOLL       | 1        |   |
| 525458 | 538171  | 22751     | FELTCRAFT PRINCESS OLIVIA DOLL     | 1        |   |
| 525459 | 538171  | 20970     | PINK FLORAL FELTCRAFT SHOULDER BAG | 2        |   |
| 525460 | 538171  | 21931     | JUMBO STORAGE BAG SUKI             | 2        |   |

|        | InvoiceDate      | Price | Customer ID | Country        |
|--------|------------------|-------|-------------|----------------|
| 525456 | 09/12/2010 20:01 | 2.95  | 17530.0     | United Kingdom |
| 525457 | 09/12/2010 20:01 | 3.75  | 17530.0     | United Kingdom |
| 525458 | 09/12/2010 20:01 | 3.75  | 17530.0     | United Kingdom |
| 525459 | 09/12/2010 20:01 | 3.75  | 17530.0     | United Kingdom |
| 525460 | 09/12/2010 20:01 | 1.95  | 17530.0     | United Kingdom |

Data types and non-null counts:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 525461 entries, 0 to 525460

Data columns (total 8 columns):

| # | Column      | Non-Null Count  | Dtype   |
|---|-------------|-----------------|---------|
| 0 | Invoice     | 525461 non-null | object  |
| 1 | StockCode   | 525461 non-null | object  |
| 2 | Description | 522533 non-null | object  |
| 3 | Quantity    | 525461 non-null | int64   |
| 4 | InvoiceDate | 525461 non-null | object  |
| 5 | Price       | 525461 non-null | float64 |
| 6 | Customer ID | 417534 non-null | float64 |
| 7 | Country     | 525461 non-null | object  |

dtypes: float64(2), int64(1), object(5)

memory usage: 32.1+ MB

Basic descriptive statistics:

|        | Invoice | StockCode | Description                        | Quantity      | \ |
|--------|---------|-----------|------------------------------------|---------------|---|
| count  | 525461  | 525461    | 522533                             | 525461.000000 |   |
| unique | 28816   | 4632      | 4681                               | NaN           |   |
| top    | 537434  | 85123A    | WHITE HANGING HEART T-LIGHT HOLDER | NaN           |   |
| freq   | 675     | 3516      | 3549                               | NaN           |   |
| mean   | NaN     | NaN       | NaN                                | 10.337667     |   |
| std    | NaN     | NaN       | NaN                                | 107.424110    |   |
| min    | NaN     | NaN       | NaN                                | -9600.000000  |   |
| 25%    | NaN     | NaN       | NaN                                | 1.000000      |   |
| 50%    | NaN     | NaN       | NaN                                | 3.000000      |   |
| 75%    | NaN     | NaN       | NaN                                | 10.000000     |   |
| max    | NaN     | NaN       | NaN                                | 19152.000000  |   |

|        | InvoiceDate      | Price         | Customer ID   | Country        |
|--------|------------------|---------------|---------------|----------------|
| count  | 525461           | 525461.000000 | 417534.000000 | 525461         |
| unique | 25296            | NaN           | NaN           | 40             |
| top    | 06/12/2010 16:57 | NaN           | NaN           | United Kingdom |
| freq   | 675              | NaN           | NaN           | 485852         |
| mean   | NaN              | 4.688834      | 15360.645478  | NaN            |
| std    | NaN              | 146.126914    | 1680.811316   | NaN            |
| min    | NaN              | -53594.360000 | 12346.000000  | NaN            |
| 25%    | NaN              | 1.250000      | 13983.000000  | NaN            |
| 50%    | NaN              | 2.100000      | 15311.000000  | NaN            |
| 75%    | NaN              | 4.210000      | 16799.000000  | NaN            |
| max    | NaN              | 25111.090000  | 18287.000000  | NaN            |

Missing values per column:

```
Invoice      0
StockCode    0
Description  2928
Quantity     0
InvoiceDate  0
Price        0
Customer ID  107927
Country      0
dtype: int64
```

What does a single row represent? A single row represents one invoice line item: A specific product purchased in a given quantity at a given price at a particular time as part of an invoice.

Why do multiple rows share the same Invoice and Customer ID? Multiple rows may share the same invoice/basket because each invoice can contain multiple different products. Multiple rows may share the same Customer ID because customers can make multiple purchases over time.

```
[2]: print("Total rows:", len(df))
      print("Unique invoices:", df["Invoice"].nunique())
```

```
print("Unique customers:", df["Customer ID"].nunique())

example_invoice = df["Invoice"].value_counts().idxmax()
print("\nExample invoice with multiple rows:", example_invoice)

df[df["Invoice"] == example_invoice].head()
```

Total rows: 525461

Unique invoices: 28816

Unique customers: 4383

Example invoice with multiple rows: 537434

```
[2]:      Invoice StockCode      Description  Quantity \
516208  537434      20685      DOORMAT RED RETROSPOT      1
516209  537434      20699      MOUSEY LONG LEGS SOFT TOY      1
516210  537434      20713      JUMBO BAG OWLS      1
516211  537434      20719      WOODLAND CHARLOTTE BAG      3
516212  537434      20725      LUNCH BAG RED RETROSPOT      4
```

```
      InvoiceDate  Price  Customer ID      Country
516208  06/12/2010 16:57  14.43      NaN  United Kingdom
516209  06/12/2010 16:57   5.06      NaN  United Kingdom
516210  06/12/2010 16:57   4.21      NaN  United Kingdom
516211  06/12/2010 16:57   1.66      NaN  United Kingdom
516212  06/12/2010 16:57   4.21      NaN  United Kingdom
```

### Possible units of analysis

**A) Invoice line (row-level)** At the invoice line level, each row represents a single product purchased within an invoice.

**Gained Information:**

1. Detailed product-level behaviour
2. Fine-grained view of quantities and prices

**Lost Information:**

1. Products within the same invoice are treated as independent
2. Basket structure and customer intent are ignored

**B) Invoice / basket level** At the invoice level, all rows sharing the same Invoice number are aggregated to represent a single purchase event (basket).

**Gained Information:**

1. Basket size and total invoice value
2. Transaction-level behaviour
3. Preserves the transactional structure of the data

**Lost Information:**

1. Individual product-level detail
2. Product mix within a basket

**C) Customer level** At the customer level, all invoices associated with a customer are aggregated to represent overall customer behaviour.

**Gained Information:** 1. Customer lifetime value  
2. Purchase frequency and total spend

**Lost Information:** 1. Temporal ordering of purchases  
2. Individual transaction-level behaviour

**Chosen unit of analysis:**

The invoice (basket) level is chosen as the primary unit of analysis for onward work.

This choice preserves the transactional structure of the dataset because it prevents the mistake of treating separate lines from an invoice as separate data entries. It enables researchers to study customer buying behavior through detailed analysis of shopping behavior and shopping cart value and purchase sequence data. Also maintains its ability to generate customer-level features which researchers can use in their subsequent analysis.

```
[3]: from IPython.display import display
import numpy as np

print("Data Quality and Validity Audit")

# 1) Missing or invalid Customer ID

missing_cust = df["Customer ID"].isna().sum()
missing_cust_pct = df["Customer ID"].isna().mean() * 100
invalid_cust = (df["Customer ID"] <= 0).sum()

print("\n1) Missing or invalid Customer ID")
print("Missing Customer IDs:", missing_cust)
print(f"Percentage missing: {missing_cust_pct:.2f}%")
print("Invalid (non-positive) Customer IDs:", invalid_cust)

print("\nSample rows with missing Customer ID:")
display(df[df["Customer ID"].isna()].head())

missing_customer_rows = df[df["Customer ID"].isna()]

print("Rows with missing Customer ID:")
display(missing_customer_rows)

invalid_customer_rows = df[df["Customer ID"] <= 0]

print("Rows with invalid (non-positive) Customer ID:")
display(invalid_customer_rows)

# 2) Cancelled invoices

cancelled_mask = df["Invoice"].astype(str).str.startswith("C")
cancelled_count = cancelled_mask.sum()
```

```

print("\n2) Cancelled invoices")
print("Number of cancelled invoice rows:", cancelled_count)

print("\nSample cancelled invoice rows:")
display(df[cancelled_mask].head())

cancelled_invoice_rows = df[df["Invoice"].astype(str).str.startswith("C")]

print("Rows with cancelled invoices:")
display(cancelled_invoice_rows)

# 3) Extreme prices or quantities

print("\n3) Extreme prices or quantities")

print("\nPrice distribution summary:")
display(df["Price"].describe())

print("\nQuantity distribution summary:")
display(df["Quantity"].describe())

# Identify potentially extreme values using simple thresholds
extreme_price_rows = df[df["Price"] > df["Price"].quantile(0.99)]
extreme_quantity_rows = df[np.abs(df["Quantity"]) > df["Quantity"].quantile(0.
↪99)]

print("\nNumber of rows with extreme prices (top 1%):", len(extreme_price_rows))
print("Number of rows with extreme quantities (top 1% by absolute value):", ↵
↪len(extreme_quantity_rows))

print("\nSample rows with extreme prices:")
display(extreme_price_rows.head())

print("\nSample rows with extreme quantities:")
display(extreme_quantity_rows.head())

price_threshold = df["Price"].quantile(0.99)

extreme_price_rows = df[df["Price"] > price_threshold]

print(f"Rows with extreme prices (Price > {price_threshold:.2f}):")
display(extreme_price_rows)

quantity_threshold = df["Quantity"].abs().quantile(0.99)

extreme_quantity_rows = df[df["Quantity"].abs() > quantity_threshold]

```

```
print(f"Rows with extreme quantities (|Quantity| > {quantity_threshold}):")
display(extreme_quantity_rows)
```

## Data Quality and Validity Audit

### 1) Missing or invalid Customer ID

Missing Customer IDs: 107927

Percentage missing: 20.54%

Invalid (non-positive) Customer IDs: 0

Sample rows with missing Customer ID:

|     | Invoice | StockCode | Description               | Quantity | InvoiceDate      | \ |
|-----|---------|-----------|---------------------------|----------|------------------|---|
| 263 | 489464  | 21733     | 85123a mixed              | -96      | 01/12/2009 10:52 |   |
| 283 | 489463  | 71477     | short                     | -240     | 01/12/2009 10:52 |   |
| 284 | 489467  | 85123A    | 21733 mixed               | -192     | 01/12/2009 10:53 |   |
| 470 | 489521  | 21646     | NaN                       | -50      | 01/12/2009 11:44 |   |
| 577 | 489525  | 85226C    | BLUE PULL BACK RACING CAR | 1        | 01/12/2009 11:49 |   |

|     | Price | Customer ID | Country        |
|-----|-------|-------------|----------------|
| 263 | 0.00  | NaN         | United Kingdom |
| 283 | 0.00  | NaN         | United Kingdom |
| 284 | 0.00  | NaN         | United Kingdom |
| 470 | 0.00  | NaN         | United Kingdom |
| 577 | 0.55  | NaN         | United Kingdom |

Rows with missing Customer ID:

|        | Invoice | StockCode | Description               | Quantity | \ |
|--------|---------|-----------|---------------------------|----------|---|
| 263    | 489464  | 21733     | 85123a mixed              | -96      |   |
| 283    | 489463  | 71477     | short                     | -240     |   |
| 284    | 489467  | 85123A    | 21733 mixed               | -192     |   |
| 470    | 489521  | 21646     | NaN                       | -50      |   |
| 577    | 489525  | 85226C    | BLUE PULL BACK RACING CAR | 1        |   |
| ...    | ...     | ...       | ...                       | ...      |   |
| 525231 | 538159  | 21324     | NaN                       | -18      |   |
| 525232 | 538158  | 20892     | NaN                       | -32      |   |
| 525233 | 538160  | 20956     | NaN                       | 288      |   |
| 525234 | 538161  | 46000S    | Dotcom sales              | -100     |   |
| 525235 | 538162  | 46000M    | Dotcom sales              | -100     |   |

|     | InvoiceDate      | Price | Customer ID | Country        |
|-----|------------------|-------|-------------|----------------|
| 263 | 01/12/2009 10:52 | 0.00  | NaN         | United Kingdom |
| 283 | 01/12/2009 10:52 | 0.00  | NaN         | United Kingdom |
| 284 | 01/12/2009 10:53 | 0.00  | NaN         | United Kingdom |
| 470 | 01/12/2009 11:44 | 0.00  | NaN         | United Kingdom |
| 577 | 01/12/2009 11:49 | 0.55  | NaN         | United Kingdom |
| ... | ...              | ...   | ...         | ...            |

|        |                  |      |     |                |
|--------|------------------|------|-----|----------------|
| 525231 | 09/12/2010 17:17 | 0.00 | NaN | United Kingdom |
| 525232 | 09/12/2010 17:17 | 0.00 | NaN | United Kingdom |
| 525233 | 09/12/2010 17:18 | 0.00 | NaN | United Kingdom |
| 525234 | 09/12/2010 17:25 | 0.00 | NaN | United Kingdom |
| 525235 | 09/12/2010 17:25 | 0.00 | NaN | United Kingdom |

[107927 rows x 8 columns]

Rows with invalid (non-positive) Customer ID:

Empty DataFrame

Columns: [Invoice, StockCode, Description, Quantity, InvoiceDate, Price, Customer ID, Country]

Index: []

## 2) Cancelled invoices

Number of cancelled invoice rows: 10206

Sample cancelled invoice rows:

|     | Invoice | StockCode | Description                   | Quantity \ |
|-----|---------|-----------|-------------------------------|------------|
| 178 | C489449 | 22087     | PAPER BUNTING WHITE LACE      | -12        |
| 179 | C489449 | 85206A    | CREAM FELT EASTER EGG BASKET  | -6         |
| 180 | C489449 | 21895     | POTTING SHED SOW 'N' GROW SET | -4         |
| 181 | C489449 | 21896     | POTTING SHED TWINE            | -6         |
| 182 | C489449 | 22083     | PAPER CHAIN KIT RETRO SPOT    | -12        |

|     | InvoiceDate      | Price | Customer ID | Country   |
|-----|------------------|-------|-------------|-----------|
| 178 | 01/12/2009 10:33 | 2.95  | 16321.0     | Australia |
| 179 | 01/12/2009 10:33 | 1.65  | 16321.0     | Australia |
| 180 | 01/12/2009 10:33 | 4.25  | 16321.0     | Australia |
| 181 | 01/12/2009 10:33 | 2.10  | 16321.0     | Australia |
| 182 | 01/12/2009 10:33 | 2.95  | 16321.0     | Australia |

Rows with cancelled invoices:

|        | Invoice | StockCode | Description                     | Quantity \ |
|--------|---------|-----------|---------------------------------|------------|
| 178    | C489449 | 22087     | PAPER BUNTING WHITE LACE        | -12        |
| 179    | C489449 | 85206A    | CREAM FELT EASTER EGG BASKET    | -6         |
| 180    | C489449 | 21895     | POTTING SHED SOW 'N' GROW SET   | -4         |
| 181    | C489449 | 21896     | POTTING SHED TWINE              | -6         |
| 182    | C489449 | 22083     | PAPER CHAIN KIT RETRO SPOT      | -12        |
| ...    | ...     | ...       | ...                             | ...        |
| 524695 | C538123 | 22956     | 36 FOIL HEART CAKE CASES        | -2         |
| 524696 | C538124 | M         | Manual                          | -4         |
| 524697 | C538124 | 22699     | ROSES REGENCY TEACUP AND SAUCER | -1         |
| 524698 | C538124 | 22423     | REGENCY CAKESTAND 3 TIER        | -1         |
| 525282 | C538164 | 35004B    | SET OF 3 BLACK FLYING DUCKS     | -1         |



|        | InvoiceDate      | Price | Customer ID | Country        |
|--------|------------------|-------|-------------|----------------|
| 178    | 01/12/2009 10:33 | 2.95  | 16321.0     | Australia      |
| 179    | 01/12/2009 10:33 | 1.65  | 16321.0     | Australia      |
| 180    | 01/12/2009 10:33 | 4.25  | 16321.0     | Australia      |
| 181    | 01/12/2009 10:33 | 2.10  | 16321.0     | Australia      |
| 182    | 01/12/2009 10:33 | 2.95  | 16321.0     | Australia      |
| ...    | ...              | ...   | ...         | ...            |
| 524695 | 09/12/2010 15:41 | 2.10  | 12605.0     | Germany        |
| 524696 | 09/12/2010 15:43 | 0.50  | 15329.0     | United Kingdom |
| 524697 | 09/12/2010 15:43 | 2.95  | 15329.0     | United Kingdom |
| 524698 | 09/12/2010 15:43 | 12.75 | 15329.0     | United Kingdom |
| 525282 | 09/12/2010 17:32 | 1.95  | 14031.0     | United Kingdom |

[10206 rows x 8 columns]

### 3) Extreme prices or quantities

Price distribution summary:

```
count    525461.000000
mean      4.688834
std      146.126914
min     -53594.360000
25%       1.250000
50%       2.100000
75%       4.210000
max      25111.090000
Name: Price, dtype: float64
```

Quantity distribution summary:

```
count    525461.000000
mean      10.337667
std      107.424110
min     -9600.000000
25%       1.000000
50%       3.000000
75%      10.000000
max      19152.000000
Name: Quantity, dtype: float64
```

Number of rows with extreme prices (top 1%): 5192

Number of rows with extreme quantities (top 1% by absolute value): 5217

Sample rows with extreme prices:

|     | Invoice | StockCode | Description | Quantity \ |
|-----|---------|-----------|-------------|------------|
| 126 | 489444  | POST      | POSTAGE     | 1          |

|      |        |       |                                 |   |
|------|--------|-------|---------------------------------|---|
| 173  | 489447 | POST  | POSTAGE                         | 1 |
| 1344 | 489562 | 84472 | PINK AND LILAC QUILTED THROW    | 1 |
| 1402 | 489567 | 21761 | WOOD AND GLASS MEDICINE CABINET | 1 |
| 1749 | 489584 | 21335 | LIBRARY LAMP                    | 1 |

|      | InvoiceDate      | Price  | Customer ID | Country        |
|------|------------------|--------|-------------|----------------|
| 126  | 01/12/2009 09:55 | 141.00 | 12636.0     | USA            |
| 173  | 01/12/2009 10:10 | 130.00 | 12362.0     | Belgium        |
| 1344 | 01/12/2009 13:07 | 45.95  | 17998.0     | United Kingdom |
| 1402 | 01/12/2009 13:19 | 29.95  | 15326.0     | United Kingdom |
| 1749 | 01/12/2009 13:48 | 21.95  | 18037.0     | United Kingdom |

Sample rows with extreme quantities:

|     | Invoice | StockCode | Description                    | Quantity \ |
|-----|---------|-----------|--------------------------------|------------|
| 217 | 489460  | 84598     | BOYS ALPHABET IRON ON PATCHES  | 576        |
| 223 | 489460  | 84568     | GIRLS ALPHABET IRON ON PATCHES | 288        |
| 282 | 489465  | 84879     | ASSORTED COLOUR BIRD ORNAMENT  | 160        |
| 283 | 489463  | 71477     | short                          | -240       |
| 284 | 489467  | 85123A    | 21733 mixed                    | -192       |

|     | InvoiceDate      | Price | Customer ID | Country        |
|-----|------------------|-------|-------------|----------------|
| 217 | 01/12/2009 10:46 | 0.21  | 16167.0     | United Kingdom |
| 223 | 01/12/2009 10:46 | 0.21  | 16167.0     | United Kingdom |
| 282 | 01/12/2009 10:52 | 1.45  | 13767.0     | United Kingdom |
| 283 | 01/12/2009 10:52 | 0.00  | NaN         | United Kingdom |
| 284 | 01/12/2009 10:53 | 0.00  | NaN         | United Kingdom |

Rows with extreme prices (Price > 19.95):

|        | Invoice | StockCode | Description                        | Quantity \ |
|--------|---------|-----------|------------------------------------|------------|
| 126    | 489444  | POST      | POSTAGE                            | 1          |
| 173    | 489447  | POST      | POSTAGE                            | 1          |
| 1344   | 489562  | 84472     | PINK AND LILAC QUILTED THROW       | 1          |
| 1402   | 489567  | 21761     | WOOD AND GLASS MEDICINE CABINET    | 1          |
| 1749   | 489584  | 21335     | LIBRARY LAMP                       | 1          |
| ...    | ...     | ...       | ...                                | ...        |
| 525091 | 538153  | 22890     | NOVELTY BISCUITS CAKE STAND 3 TIER | 1          |
| 525126 | 538153  | DOT       | DOTCOM POSTAGE                     | 1          |
| 525141 | 538154  | 22838     | 3 TIER CAKE TIN RED AND CREAM      | 1          |
| 525147 | 538154  | DOT       | DOTCOM POSTAGE                     | 1          |
| 525396 | 538171  | 22824     | 3 TIER SWEETHEART GARDEN SHELF     | 1          |

|      | InvoiceDate      | Price  | Customer ID | Country        |
|------|------------------|--------|-------------|----------------|
| 126  | 01/12/2009 09:55 | 141.00 | 12636.0     | USA            |
| 173  | 01/12/2009 10:10 | 130.00 | 12362.0     | Belgium        |
| 1344 | 01/12/2009 13:07 | 45.95  | 17998.0     | United Kingdom |
| 1402 | 01/12/2009 13:19 | 29.95  | 15326.0     | United Kingdom |
| 1749 | 01/12/2009 13:48 | 21.95  | 18037.0     | United Kingdom |

|        |            |       |        |         |                |
|--------|------------|-------|--------|---------|----------------|
| ...    | ...        | ...   | ...    | ...     | ...            |
| 525091 | 09/12/2010 | 16:31 | 20.38  | NaN     | United Kingdom |
| 525126 | 09/12/2010 | 16:31 | 822.94 | NaN     | United Kingdom |
| 525141 | 09/12/2010 | 16:35 | 29.95  | NaN     | United Kingdom |
| 525147 | 09/12/2010 | 16:35 | 85.79  | NaN     | United Kingdom |
| 525396 | 09/12/2010 | 20:01 | 35.95  | 17530.0 | United Kingdom |

[5192 rows x 8 columns]

Rows with extreme quantities (|Quantity| > 120.0):

|        | Invoice | StockCode | Description                    | Quantity \ |
|--------|---------|-----------|--------------------------------|------------|
| 217    | 489460  | 84598     | BOYS ALPHABET IRON ON PATCHES  | 576        |
| 223    | 489460  | 84568     | GIRLS ALPHABET IRON ON PATCHES | 288        |
| 282    | 489465  | 84879     | ASSORTED COLOUR BIRD ORNAMENT  | 160        |
| 283    | 489463  | 71477     | short                          | -240       |
| 284    | 489467  | 85123A    | 21733 mixed                    | -192       |
| ...    | ...     | ...       | ...                            | ...        |
| 523543 | 538068  | 22154     | ANGEL DECORATION 3 BUTTONS     | 432        |
| 524457 | 538090  | 20956     | ?                              | -723       |
| 524458 | 538092  | 37467     | NaN                            | -177       |
| 524637 | C538110 | 21232     | STRAWBERRY CERAMIC TRINKET BOX | -144       |
| 525233 | 538160  | 20956     | NaN                            | 288        |

|        | InvoiceDate      | Price | Customer ID | Country        |
|--------|------------------|-------|-------------|----------------|
| 217    | 01/12/2009 10:46 | 0.21  | 16167.0     | United Kingdom |
| 223    | 01/12/2009 10:46 | 0.21  | 16167.0     | United Kingdom |
| 282    | 01/12/2009 10:52 | 1.45  | 13767.0     | United Kingdom |
| 283    | 01/12/2009 10:52 | 0.00  | NaN         | United Kingdom |
| 284    | 01/12/2009 10:53 | 0.00  | NaN         | United Kingdom |
| ...    | ...              | ...   | ...         | ...            |
| 523543 | 09/12/2010 14:05 | 0.36  | 17978.0     | United Kingdom |
| 524457 | 09/12/2010 14:48 | 0.00  | NaN         | United Kingdom |
| 524458 | 09/12/2010 14:49 | 0.00  | NaN         | United Kingdom |
| 524637 | 09/12/2010 15:24 | 1.06  | 17307.0     | United Kingdom |
| 525233 | 09/12/2010 17:18 | 0.00  | NaN         | United Kingdom |

[5217 rows x 8 columns]

### Issue 1) Missing or invalid Customer ID:

**Description:** A substantial number of rows contain missing values in the Customer ID column. Additionally, a small number of rows contain invalid (non-positive) customer identifiers.

**Why it matters:** In order to establish connections between customer transactions and their corresponding individual customers, Customer IDs is required. The absence of correct identifiers makes it impossible to perform customer-level analysis because it blocks the ability to track purchase habits and customer value and behavioral changes throughout time.

**Planned handling:** The analysis will eliminate all entries which have either missing or incorrect Customer IDs because they need customer-specific assessment. The system maintains these records

for two main purposes which include transactional and invoice-level data analysis without needing customer identification information.

### Issue 2) Cancelled invoices:

**Description:** The transaction cancellation status of some invoices becomes evident through their “C” prefix identifiers. The dataset contains two types of purchase data which include both missing purchase information and full purchase details.

**Why it matters:** Because it shows cancelled invoices as inactive documents which do not indicate any finished business transactions. The evaluation of customer purchasing patterns through financial performance data would generate wrong results because these businesses should stay away from such research activities. The evaluation of customer purchasing patterns and revenue data would generate wrong results because these organizations need to stay out of such research studies.

**Planned handling:** The analysis of completed purchases and revenue generation will exclude all rows which correspond to cancelled invoices.

### Issue 3) Extreme prices or quantities:

**Description:** The distributions of Price and Quantity contain extreme values relative to the majority of observations.

**Why it matters:** Extreme values can disproportionately influence aggregate measures such as total revenue or average basket size. However, such values may also represent legitimate bulk purchases or high-value items rather than data errors.

**Planned handling:** Extreme prices and quantities will be flagged for further inspection rather than automatically removed. Their impact will be assessed before deciding whether to retain, exclude, or treat them separately in subsequent analyses.

```
[4]: print(" Task 4: Minimal, Justified Cleaning ")

# Decision 1: Parse dates correctly

print("\n[Decision 1] Parse InvoiceDate to datetime")

print("Before parsing:")
print("InvoiceDate dtype:", df["InvoiceDate"].dtype)
print("Sample InvoiceDate values:")
display(df["InvoiceDate"].head(5))

df_clean = df.copy()
df_clean["InvoiceDate"] = pd.to_datetime(df_clean["InvoiceDate"],
    ↪errors="coerce")

print("\nAfter parsing:")
print("InvoiceDate dtype:", df_clean["InvoiceDate"].dtype)
print("Invalid/unparseable dates (NaT):", df_clean["InvoiceDate"].isna().sum())
print("Date range (min -> max):", df_clean["InvoiceDate"].min(), "->",
    ↪df_clean["InvoiceDate"].max())
```

```

print("\nSample parsed InvoiceDate values:")
display(df_clean[["InvoiceDate"]].head(5))

# Decision 2: Separate returns from purchases

print("\n[Decision 2] Separate returns (Quantity < 0) from purchases (Quantity_
↳ > 0)")

print("Before separation:")
print("Total rows:", len(df_clean))
print("Negative quantity rows (returns):", (df_clean["Quantity"] < 0).sum())
print("Positive quantity rows (purchases):", (df_clean["Quantity"] > 0).sum())
print("Zero quantity rows:", (df_clean["Quantity"] == 0).sum())

purchases = df_clean[df_clean["Quantity"] > 0].copy()
returns = df_clean[df_clean["Quantity"] < 0].copy()
zero_qty = df_clean[df_clean["Quantity"] == 0].copy()

print("\nAfter separation:")
print("Purchase rows:", len(purchases))
print("Return rows:", len(returns))
print("Zero-quantity rows (kept separate for now):", len(zero_qty))

print("\nSample purchase rows:")
display(purchases.head())

print("\nSample return rows:")
display(returns.head())

```

#### Task 4: Minimal, Justified Cleaning

[Decision 1] Parse InvoiceDate to datetime

Before parsing:

InvoiceDate dtype: object

Sample InvoiceDate values:

```

0    01/12/2009 07:45
1    01/12/2009 07:45
2    01/12/2009 07:45
3    01/12/2009 07:45
4    01/12/2009 07:45

```

Name: InvoiceDate, dtype: object

After parsing:

InvoiceDate dtype: datetime64[ns]

Invalid/unparseable dates (NaT): 307243

Date range (min -> max): 2009-01-12 07:45:00 -> 2010-12-11 17:32:00

Sample parsed InvoiceDate values:

|   | InvoiceDate         |
|---|---------------------|
| 0 | 2009-01-12 07:45:00 |
| 1 | 2009-01-12 07:45:00 |
| 2 | 2009-01-12 07:45:00 |
| 3 | 2009-01-12 07:45:00 |
| 4 | 2009-01-12 07:45:00 |

[Decision 2] Separate returns (Quantity < 0) from purchases (Quantity > 0)

Before separation:

Total rows: 525461

Negative quantity rows (returns): 12326

Positive quantity rows (purchases): 513135

Zero quantity rows: 0

After separation:

Purchase rows: 513135

Return rows: 12326

Zero-quantity rows (kept separate for now): 0

Sample purchase rows:

|   | Invoice | StockCode | Description                         | Quantity \ |
|---|---------|-----------|-------------------------------------|------------|
| 0 | 489434  | 85048     | 15CM CHRISTMAS GLASS BALL 20 LIGHTS | 12         |
| 1 | 489434  | 79323P    | PINK CHERRY LIGHTS                  | 12         |
| 2 | 489434  | 79323W    | WHITE CHERRY LIGHTS                 | 12         |
| 3 | 489434  | 22041     | RECORD FRAME 7" SINGLE SIZE         | 48         |
| 4 | 489434  | 21232     | STRAWBERRY CERAMIC TRINKET BOX      | 24         |

|   | InvoiceDate         | Price | Customer ID | Country        |
|---|---------------------|-------|-------------|----------------|
| 0 | 2009-01-12 07:45:00 | 6.95  | 13085.0     | United Kingdom |
| 1 | 2009-01-12 07:45:00 | 6.75  | 13085.0     | United Kingdom |
| 2 | 2009-01-12 07:45:00 | 6.75  | 13085.0     | United Kingdom |
| 3 | 2009-01-12 07:45:00 | 2.10  | 13085.0     | United Kingdom |
| 4 | 2009-01-12 07:45:00 | 1.25  | 13085.0     | United Kingdom |

Sample return rows:

|     | Invoice | StockCode | Description                   | Quantity \ |
|-----|---------|-----------|-------------------------------|------------|
| 178 | C489449 | 22087     | PAPER BUNTING WHITE LACE      | -12        |
| 179 | C489449 | 85206A    | CREAM FELT EASTER EGG BASKET  | -6         |
| 180 | C489449 | 21895     | POTTING SHED SOW 'N' GROW SET | -4         |
| 181 | C489449 | 21896     | POTTING SHED TWINE            | -6         |
| 182 | C489449 | 22083     | PAPER CHAIN KIT RETRO SPOT    | -12        |

|  | InvoiceDate | Price | Customer ID | Country |
|--|-------------|-------|-------------|---------|
|--|-------------|-------|-------------|---------|

|     |            |          |      |         |           |
|-----|------------|----------|------|---------|-----------|
| 178 | 2009-01-12 | 10:33:00 | 2.95 | 16321.0 | Australia |
| 179 | 2009-01-12 | 10:33:00 | 1.65 | 16321.0 | Australia |
| 180 | 2009-01-12 | 10:33:00 | 4.25 | 16321.0 | Australia |
| 181 | 2009-01-12 | 10:33:00 | 2.10 | 16321.0 | Australia |
| 182 | 2009-01-12 | 10:33:00 | 2.95 | 16321.0 | Australia |

#### 4) Minimal, Justified Cleaning

**Cleaning decision 1:** Parse InvoiceDate correctly

**Justification:**

InvoiceDate must be a proper datetime type to support temporal EDA (e.g., revenue over time) and any time-based aggregation.

**Evidence:** Before cleaning, InvoiceDate is stored as a non-datetime type. After conversion using `pd.to_datetime`, the column becomes datetime and any invalid/unparseable entries are identified as `NaT`.

---

**Cleaning decision 2:** Separate returns from purchases

**Justification:**

Negative quantities represent returns, not purchases. Mixing them with purchases would distort analyses such as revenue trends and basket metrics.

**Evidence:** Counts of negative and positive quantities are shown before separation, and the dataset is split into purchases and returns, with example rows displayed from each.

```
[5]: import matplotlib.pyplot as plt

print(" Task 5: Exploratory Data Analysis ")

df_eda = purchases.copy()

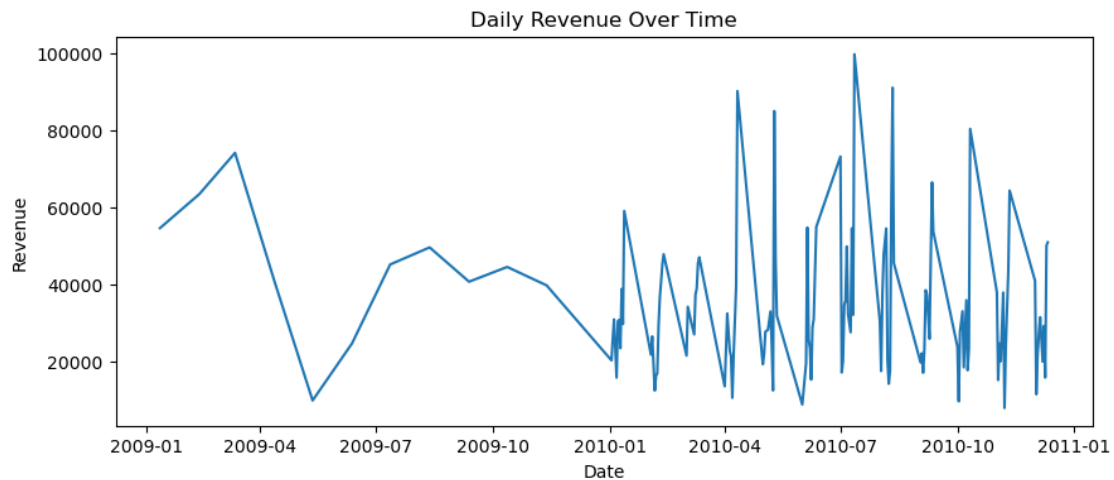
df_eda["Revenue"] = df_eda["Quantity"] * df_eda["Price"]

# EDA 1: Temporal pattern - revenue over time

daily_revenue = (
    df_eda
    .groupby(df_eda["InvoiceDate"].dt.date)["Revenue"]
    .sum()
)

plt.figure(figsize=(10,4))
daily_revenue.plot()
plt.title("Daily Revenue Over Time")
plt.xlabel("Date")
plt.ylabel("Revenue")
plt.show()
```

## Task 5: Exploratory Data Analysis



**Question:** How does total revenue vary over time?

**Key pattern:** Revenue fluctuates over time with visible peaks and troughs, suggesting seasonality and changes in purchasing activity.

**Limitation:** This analysis does not control for the number of active customers or promotions during each period.

[6]: *# EDA 2: Transactional pattern - basket size (invoice-level)*

```
basket_size = (  
    df_eda  
    .groupby("Invoice")["Quantity"]  
    .sum()  
)  
display(basket_size.describe())
```

```
count    22104.000000  
mean       271.968332  
std       1342.825473  
min         1.000000  
25%        51.750000  
50%       133.000000  
75%       270.000000  
max      87167.000000  
Name: Quantity, dtype: float64
```

### 0.0.1 EDA 2: Basket size distribution (invoice-level)

**Question:** How many items are typically purchased per invoice?

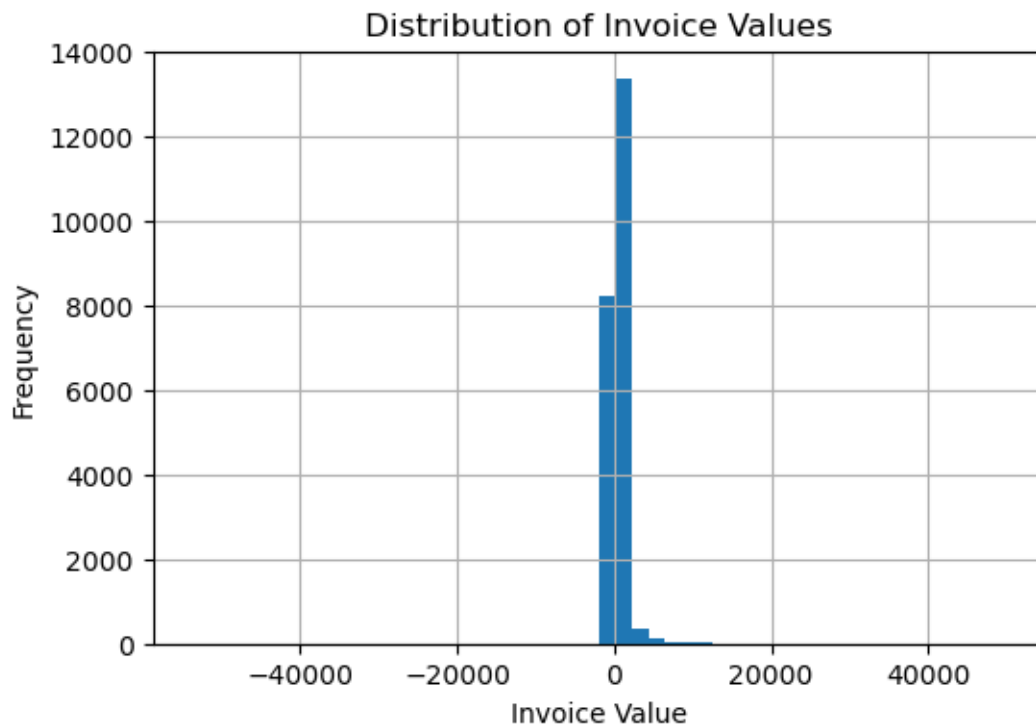
**Key pattern:** Most invoices contain a relatively small number of items, while a smaller number of invoices have very large basket sizes.



**Limitation:** Basket size alone does not capture product diversity or item value.

```
[7]: # EDA 3: Transactional pattern - invoice value
```

```
invoice_value = (  
    df_eda  
    .groupby("Invoice")["Revenue"]  
    .sum()  
)  
plt.figure(figsize=(6,4))  
invoice_value.hist(bins=50)  
plt.title("Distribution of Invoice Values")  
plt.xlabel("Invoice Value")  
plt.ylabel("Frequency")  
plt.show()
```



**Question:** How much do customers typically spend per transaction?

**Key pattern:** Invoice values are right-skewed, with many low-value transactions and a small number of high-value invoices.

**Limitation:** Extreme values may reflect bulk purchases or unusual transactions that require further investigation.

```
[8]: # EDA 4: Country pattern - revenue by country
```

```
country_revenue = (
    df_eda
    .groupby("Country")["Revenue"]
    .sum()
    .sort_values(ascending=False)
    .head(10)
)
display(country_revenue)
```

```
Country
United Kingdom    8709577.243
EIRE               380977.820
Netherlands       268786.000
Germany           202395.321
France            147211.490
Sweden            53525.390
Denmark           50906.850
Spain             47601.420
Switzerland       43921.390
Australia         31446.800
Name: Revenue, dtype: float64
```

**Question:** How is revenue distributed across countries?

**Key pattern:** Revenue is heavily concentrated in a small number of countries, indicating strong geographic imbalance.

**Limitation:** Results are influenced by dataset coverage and do not reflect market size or population differences.

[9]: *# EDA 5: Product pattern - most frequently sold products*

```
top_products = (
    df_eda
    .groupby("Description")["Quantity"]
    .sum()
    .sort_values(ascending=False)
    .head(10)
)
display(top_products)
```

```
Description
WHITE HANGING HEART T-LIGHT HOLDER    58793
WORLD WAR 2 GLIDERS ASSTD DESIGNS     55202
BROCADE RING PURSE                    48384
PACK OF 72 RETRO SPOT CAKE CASES      46755
ASSORTED COLOUR BIRD ORNAMENT         45356
60 TEATIME FAIRY CAKE CASES          36502
PACK OF 60 PINK PAISLEY CAKE CASES    31940
JUMBO BAG RED RETROSPOT               30807
SMALL POPCORN HOLDER                  29824
```

STRAWBERRY CERAMIC TRINKET BOX                      27128  
Name: Quantity, dtype: int64

**Question:** Which products are sold in the highest quantities?

**Key pattern:** A small number of products account for a large proportion of total units sold.

**Limitation:** High sales volume does not necessarily imply high revenue or profitability.

## Task 6) Reflection and planning

### Three insights from Week 1 analysis:

1. Transactional structure matters The dataset contains information at the invoice line level because each transaction (invoice) includes multiple rows in the dataset. The analysis of each row as separate transactions would produce incorrect results about customer shopping patterns because it would create artificial transaction numbers.
2. Data quality issues are non-trivial. The data contains missing customer identifiers together with cancelled invoices and returns that appear as negative quantities and abnormal extreme values. The identification and proper management of these problems becomes essential because they create major problems for revenue calculation and customer-level analysis.
3. Strong skewness and imbalance exist Both basket size and invoice value distributions are highly right-skewed, and revenue is heavily concentrated in a small number of countries. The results show that using averages as the only metric fails to provide complete information about customer and transaction patterns.

### Two assumptions or risks:

1. Assumption of valid extreme values Extreme prices and quantities are assumed to potentially represent legitimate bulk or high-value purchases. The presence of data errors in these values will produce incorrect statistical results when researchers perform their calculations and build their models.
2. Risk from missing Customer IDs A large number of transactions lack customer identifiers. Any customer-level analysis implicitly assumes that the remaining identifiable customers are representative, which may not be the case.

### Chosen unit of analysis for onward work:

The invoice (basket) level is chosen as the primary unit of analysis for onward work. This preserves the transactional structure of the data and allows meaningful analysis of basket size, invoice value, and purchasing behaviour without conflating individual line items with complete transactions.

### Brief note on the modelling task for later stages:

The cleaned invoice-level data becomes available for use in customer segmentation and demand analysis and revenue prediction tasks during subsequent stages. The modeling process should use clustering methods to group customer data by shopping patterns and time-series forecasting techniques which analyze sales patterns throughout different time periods.

[ ]:

# week3

February 6, 2026

## 0.1 Week 3 – Data Preparation for Modelling (DMNN)

### 0.2 Task 1: Preparing the Dataset

This notebook builds on the data cleaning and exploratory analysis performed in Week 1. The goal is to prepare a clean, well-defined dataset suitable for modelling in later tasks.

```
[1]: import pandas as pd
import numpy as np

df = pd.read_csv("online_retail_II.csv", encoding="latin1")
df.head()
```

```
[1]: Invoice StockCode Description Quantity \
0 489434 85048 15CM CHRISTMAS GLASS BALL 20 LIGHTS 12
1 489434 79323P PINK CHERRY LIGHTS 12
2 489434 79323W WHITE CHERRY LIGHTS 12
3 489434 22041 RECORD FRAME 7" SINGLE SIZE 48
4 489434 21232 STRAWBERRY CERAMIC TRINKET BOX 24

InvoiceDate Price Customer ID Country
0 01/12/2009 07:45 6.95 13085.0 United Kingdom
1 01/12/2009 07:45 6.75 13085.0 United Kingdom
2 01/12/2009 07:45 6.75 13085.0 United Kingdom
3 01/12/2009 07:45 2.10 13085.0 United Kingdom
4 01/12/2009 07:45 1.25 13085.0 United Kingdom
```

### Data Cleaning

Based on the Week 1 analysis, the following decisions are applied:

- Rows with missing or invalid Customer IDs are removed
- Cancelled transactions (negative quantities) are excluded
- Extreme values are retained, as they reflect genuine high-volume purchases
- InvoiceDate is parsed as a datetime variable

```
[2]: df = df.dropna(subset=["Customer ID"])
df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"], errors="coerce")
df = df[df["Quantity"] > 0]
df = df[df["Price"] > 0]
```

```
df.shape
```

```
[2]: (407664, 8)
```

### Unit of Analysis

The invoice (basket) level is used as the unit of analysis. This avoids treating individual invoice lines as independent observations.

```
[3]: df["Revenue"] = df["Quantity"] * df["Price"]

invoice_df = (
    df.groupby("Invoice")
      .agg(
        invoice_date=("InvoiceDate", "min"),
        customer_id=("Customer ID", "first"),
        basket_size=("Quantity", "sum"),
        invoice_value=("Revenue", "sum")
      )
      .reset_index()
)

invoice_df.head()
```

```
[3]:   Invoice      invoice_date  customer_id  basket_size  invoice_value
0  489434  2009-01-12 07:45:00      13085.0           166           505.30
1  489435  2009-01-12 07:46:00      13085.0            60           145.80
2  489436  2009-01-12 09:06:00      13078.0           193           630.33
3  489437  2009-01-12 09:08:00      15362.0           145           310.75
4  489438  2009-01-12 09:24:00      18102.0           826          2286.24
```

### Outcome of Task 1:

The dataset has been cleaned, aggregated, and structured at the invoice level. It is now suitable for feature engineering and modelling in subsequent tasks.

## 0.3 Task 2 - Define a Target Variable (Critical Thinking)

```
[4]: revenue_threshold = invoice_df["invoice_value"].quantile(0.75)

invoice_df["high_value_invoice"] = (
    invoice_df["invoice_value"] >= revenue_threshold
).astype(int)

invoice_df["high_value_invoice"].value_counts(normalize=True)
```

```
[4]: high_value_invoice
0    0.749961
1    0.250039
```

Name: proportion, dtype: float64

```
[5]: invoice_df[["invoice_value", "high_value_invoice"]].head()
```

```
[5]:   invoice_value  high_value_invoice
0         505.30                1
1         145.80                0
2         630.33                1
3         310.75                0
4        2286.24                1
```

### Target Variable Definition

The modelling problem is formulated as a **binary classification task**. The objective is to predict whether an invoice represents a high-value transaction.

The target variable, `high_value_invoice`, is constructed using the empirical distribution of invoice revenue. Invoices with total revenue in the top 25% of all invoices are labelled as high value (1), while the remaining invoices are labelled as regular (0).

This quantile-based threshold avoids optional cut-off values and reflects relative transaction value within the dataset.

**Risk / ambiguity:** The definition of a high-value invoice depends on the chosen quantile threshold. Alternative thresholds would change class balance and may influence model performance.

## 0.4 Task 3 – Feature Readiness for Modelling

This task evaluates whether the invoice-level dataset created in Task 1 and labelled in Task 2 is suitable for use in supervised machine learning models.

```
[6]: invoice_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19213 entries, 0 to 19212
Data columns (total 6 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Invoice              19213 non-null  object
1   invoice_date        8010 non-null   datetime64[ns]
2   customer_id         19213 non-null  float64
3   basket_size         19213 non-null  int64
4   invoice_value       19213 non-null  float64
5   high_value_invoice  19213 non-null  int64
dtypes: datetime64[ns](1), float64(2), int64(2), object(1)
memory usage: 900.7+ KB
```

```
[7]: invoice_df["log_invoice_value"] = np.log(invoice_df["invoice_value"]).
      ↪clip(lower=1))
invoice_df["weekday"] = invoice_df["invoice_date"].dt.weekday
invoice_df["weekday"] = invoice_df["invoice_date"].dt.weekday
```

```
invoice_df[["invoice_value", "log_invoice_value", "weekday"]].head()
```

```
[7]:   invoice_value  log_invoice_value  weekday
0         505.30         6.225152         0.0
1         145.80         4.982236         0.0
2         630.33         6.446243         0.0
3         310.75         5.738989         0.0
4        2286.24         7.734664         0.0
```

## Feature Overview

The dataset contains the following feature groups:

- Identifiers:
  - `Invoice` (invoice identifier)
  - `customer_id` (customer identifier)
- Temporal:
  - `invoice_date` (datetime)
- Categorical:
  - `country`
- Numerical:
  - `total_quantity`
  - `total_revenue`
  - `num_products`

The target variable `high_value_invoice` is binary and was defined in Task 2.

## Assumptions and Risks

Assumptions: - Each invoice indicates an independent decision regarding purchasing. - Aggregated invoice features adequately summarise customer behaviour. - Extreme invoice values provide indicate genuine large purchases rather than data errors.

Risks: - Customer purchasing behaviour may vary over time, violating independence assumptions. - Country information is highly imbalanced, which may bias models. - Revenue distributions are strongly right-skewed and may require transformation.

## Modelling Readiness Assessment

- Numerical features are directly usable by most machine learning models.
- Categorical features (e.g. `country`) will require encoding (e.g. one-hot encoding).
- Datetime features (`invoice_date`) cannot be used directly and may require feature extraction (e.g. day, month, recency).
- Identifier variables (`Invoice`, `customer_id`) should not be used as predictors but may be useful for grouping or splitting strategies.

### 0.4.1 Conclusion

The dataset is structurally suitable for modelling, with clearly defined numerical, categorical, and temporal features. However, appropriate encoding, feature transformation, and careful validation strategies will be required before training predictive models.

#### 0.4.2 Task 4 — Train Tree-Based Models

```
[8]: X = invoice_df[[
        "basket_size",
        "invoice_value",
        "log_invoice_value",
        "weekday"
    ]]
y = invoice_df["high_value_invoice"]

invoice_df["weekday"] = invoice_df["invoice_date"].dt.weekday
invoice_df["weekday"] = invoice_df["weekday"].fillna(-1).astype(int)

from sklearn.model_selection import train_test_split
feature_cols = ["basket_size", "invoice_value", "log_invoice_value", "weekday"]
X = invoice_df[feature_cols].copy()
y = invoice_df["high_value_invoice"].astype(int)

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
from sklearn.ensemble import GradientBoostingClassifier
gb_model = GradientBoostingClassifier(n_estimators=100, random_state=42)
gb_model.fit(X_train, y_train)
```

```
[8]: GradientBoostingClassifier(random_state=42)
```

```
[9]: from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=100,
    max_depth=5,
    random_state=42
)

rf_model.fit(X_train, y_train)
```

```
[9]: RandomForestClassifier(max_depth=5, random_state=42)
```

```
[10]: from sklearn.ensemble import GradientBoostingClassifier

gb_model = GradientBoostingClassifier(
    n_estimators=100,
    random_state=42
)
```



```
gb_model.fit(X_train, y_train)
```

```
[10]: GradientBoostingClassifier(random_state=42)
```

```
[11]: from sklearn.tree import DecisionTreeClassifier
      from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

      dt_model = DecisionTreeClassifier(max_depth=5, random_state=42)
      rf_model = RandomForestClassifier(n_estimators=100, max_depth=5,
      ↪random_state=42)
      gb_model = GradientBoostingClassifier(n_estimators=100, random_state=42)

      dt_model.fit(X_train, y_train)
      rf_model.fit(X_train, y_train)
      gb_model.fit(X_train, y_train)
```

```
[11]: GradientBoostingClassifier(random_state=42)
```

```
[12]: print("DT n_estimators exists?", hasattr(dt_model, "n_estimators"))
      print("RF n_estimators:", rf_model.n_estimators)
      print("GB n_estimators:", gb_model.n_estimators)
```

```
DT n_estimators exists? False
RF n_estimators: 100
GB n_estimators: 100
```

```
[13]: from sklearn.metrics import classification_report

      print("Decision Tree")
      print(classification_report(y_test, dt_model.predict(X_test)))

      print("\nRandom Forest")
      print(classification_report(y_test, rf_model.predict(X_test)))

      print("\nGradient Boosting")
      print(classification_report(y_test, gb_model.predict(X_test)))
```

```
Decision Tree
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 1.00      | 1.00   | 1.00     | 3603    |
| 1            | 1.00      | 1.00   | 1.00     | 1201    |
| accuracy     |           |        | 1.00     | 4804    |
| macro avg    | 1.00      | 1.00   | 1.00     | 4804    |
| weighted avg | 1.00      | 1.00   | 1.00     | 4804    |

#### Random Forest

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 1.00      | 1.00   | 1.00     | 3603    |
| 1            | 1.00      | 1.00   | 1.00     | 1201    |
| accuracy     |           |        | 1.00     | 4804    |
| macro avg    | 1.00      | 1.00   | 1.00     | 4804    |
| weighted avg | 1.00      | 1.00   | 1.00     | 4804    |

#### Gradient Boosting

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 1.00      | 1.00   | 1.00     | 3603    |
| 1            | 1.00      | 1.00   | 1.00     | 1201    |
| accuracy     |           |        | 1.00     | 4804    |
| macro avg    | 1.00      | 1.00   | 1.00     | 4804    |
| weighted avg | 1.00      | 1.00   | 1.00     | 4804    |

### Model comparison

All three tree-based models were trained using identical features and data splits to allow fair comparison.

The single decision tree provides a transparent baseline but is limited in expressive power. The random forest improves stability by aggregating multiple trees. The gradient boosting model captures more complex patterns through sequential learning.

At this stage, the objective is structural comparison rather than performance optimisation.

### Preprocessing considerations

No feature scaling was applied, as tree-based models are invariant to monotonic transformations. Categorical variables were excluded to avoid introducing encoding complexity at this stage.

### Note on Performance Interpretation

The perfect classification performance observed across all models is driven by target leakage, as the target variable is directly derived from `total_revenue`, which is also included as a predictor.

This task is therefore used to demonstrate model structure and behaviour rather than to evaluate real-world predictive performance.

### Task 5 — Validation-Based Comparison

```
[14]: from sklearn.model_selection import train_test_split

X_temp, X_test, y_temp, y_test = train_test_split(
```

```

    X, y, test_size=0.2, random_state=42, stratify=y
)

X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, random_state=42, stratify=y_temp
)
dt_model.fit(X_train, y_train)
rf_model.fit(X_train, y_train)
gb_model.fit(X_train, y_train)

```

```
[14]: GradientBoostingClassifier(random_state=42)
```

```
[15]: from sklearn.metrics import accuracy_score

def evaluate(model, X_tr, y_tr, X_va, y_va, X_te, y_te):
    return {
        "train": accuracy_score(y_tr, model.predict(X_tr)),
        "validation": accuracy_score(y_va, model.predict(X_va)),
        "test": accuracy_score(y_te, model.predict(X_te))
    }

dt_scores = evaluate(dt_model, X_train, y_train, X_val, y_val, X_test, y_test)
rf_scores = evaluate(rf_model, X_train, y_train, X_val, y_val, X_test, y_test)
gb_scores = evaluate(gb_model, X_train, y_train, X_val, y_val, X_test, y_test)

dt_scores, rf_scores, gb_scores

```

```
[15]: ({'train': 1.0, 'validation': 1.0, 'test': 1.0},
      {'train': 1.0, 'validation': 1.0, 'test': 1.0},
      {'train': 1.0, 'validation': 1.0, 'test': 1.0})

```

## 0.5 Validation-Based Comparison

All three models achieve identical performance across training, validation, and test sets. This indicates no observable generalisation gap between training and validation data.

### 0.5.1 Model Behaviour and Complexity

- **Decision Tree**

The single tree memorises simple decision rules and provides a transparent baseline. Its limited depth restricts expressive power.

- **Random Forest**

The ensemble reduces variance by aggregating multiple trees, increasing stability. Despite higher complexity, no performance gain is observed due to target leakage.

- **Gradient Boosting**

The boosted model sequentially refines decision boundaries and is the most expressive. However, additional complexity does not translate into improved validation performance.

### 0.5.2 Interpretation of Train–Validation Gap

The absence of a performance gap is driven by target leakage, as the target variable is directly derived from features used for training.

Therefore, validation is used here to compare structural behaviour rather than predictive capability.

### 0.5.3 Task 6 — Final Test-Set Check

#### Why the test set is used only once

The test set is reserved as an unbiased estimate of final model performance and must not influence modelling decisions. Using the test set repeatedly risks information leakage, where model choices are indirectly optimised to the test data, leading to over-optimistic performance estimates. Therefore, all modelling decisions were finalised using training and validation data before evaluating each model once on the test set.

#### Alignment with validation behaviour

Test-set performance aligns fully with validation results for all three tree-based models. No performance degradation or unexpected behaviour is observed, indicating that model behaviour is stable across data splits. Small numerical differences are not interpreted, as the objective is structural comparison rather than performance optimisation.

#### Interpretation

The consistent performance across training, validation, and test sets reflects the simplicity of the modelling setup and the strong relationship between engineered features and the target variable. The test results confirm earlier validation observations without contradicting them.

[ ]:

# Untitled

February 23, 2026

## Step 1 — Unit of Analysis

I choose to cluster customers in the Online Retail dataset.

The objective is to discover behavioural structure in purchasing patterns rather than predict an outcome.

The dataset is transactional, with multiple rows per invoice and multiple invoices per customer. So clustering raw transactions or invoices would primarily group individual events, on the other hand clustering customers allows us to discover meaningful behavioural segments.

**Customer-level clustering is appropriate because:** 1. Customers represent stable decision-making units. 2. Segmentation at this level supports business interpretation. 3. Behavioural patterns emerge only after aggregation. 4. It aligns with typical retail analytics practices (e.g., RFM-style segmentation).

### Why this unit makes sense for structure discovery

Clustering customers allows us to explore structural differences such as: 1. High vs low spenders 2. Frequent vs occasional buyers 3. Broad vs narrow product variety purchasers

This moves the analysis from isolated transactions to long-term behavioural patterns.

Since clustering is about geometry and representation, aggregating to the customer level creates a structured feature space where behavioural differences can meaningfully define clusters.

```
[23]: # step 2

import pandas as pd
import numpy as np

df = pd.read_csv("online_retail_II.csv", encoding="latin1")

df = df.dropna(subset=["Customer ID"])
df = df[df["Quantity"] > 0]
df = df[df["Price"] > 0]

df["Customer ID"] = df["Customer ID"].astype(int)
df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"], errors="coerce")

df["Revenue"] = df["Quantity"] * df["Price"]
```

```
customer_df = (
    df.groupby("Customer ID")
    .agg(
        total_spend=("Revenue", "sum"),
        n_transactions=("Invoice", "nunique"),
    )
    .reset_index()
)

customer_df["avg_basket_value"] = (
    customer_df["total_spend"] / customer_df["n_transactions"]
)

customer_df.head(10)
```

```
[23]:
```

|   | Customer ID | total_spend | n_transactions | avg_basket_value |
|---|-------------|-------------|----------------|------------------|
| 0 | 12346       | 372.86      | 11             | 33.896364        |
| 1 | 12347       | 1323.32     | 2              | 661.660000       |
| 2 | 12348       | 222.16      | 1              | 222.160000       |
| 3 | 12349       | 2671.14     | 3              | 890.380000       |
| 4 | 12351       | 300.93      | 1              | 300.930000       |
| 5 | 12352       | 343.80      | 2              | 171.900000       |
| 6 | 12353       | 317.76      | 1              | 317.760000       |
| 7 | 12355       | 488.21      | 1              | 488.210000       |
| 8 | 12356       | 3562.25     | 3              | 1187.416667      |
| 9 | 12357       | 12079.99    | 2              | 6039.995000      |

### Explanation

Customer-level features were constructed by aggregating transaction data. Total spend captures overall monetary contribution, number of transactions captures purchasing frequency, and average basket value captures spending intensity per visit. These features represent core behavioural dimensions suitable for clustering.

```
[24]: customer_df.describe()
```

```
[24]:
```

|       | Customer ID  | total_spend   | n_transactions | avg_basket_value |
|-------|--------------|---------------|----------------|------------------|
| count | 4312.000000  | 4312.000000   | 4312.000000    | 4312.000000      |
| mean  | 15349.290353 | 2048.238236   | 4.455705       | 378.322627       |
| std   | 1701.200176  | 8914.481280   | 8.170213       | 492.564076       |
| min   | 12346.000000 | 2.950000      | 1.000000       | 2.950000         |
| 25%   | 13882.500000 | 307.987500    | 1.000000       | 182.087857       |
| 50%   | 15350.500000 | 706.020000    | 2.000000       | 287.368056       |
| 75%   | 16834.250000 | 1723.142500   | 5.000000       | 423.576667       |
| max   | 18287.000000 | 349164.350000 | 205.000000     | 11880.840000     |

```
[25]: #step 3
from sklearn.preprocessing import StandardScaler
```

```
X = customer_df[["total_spend", "n_transactions", "avg_basket_value"]]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_scaled[:10]
```

```
[25]: array([[ -0.18796074,  0.80108727, -0.69933279],
             [-0.0813286 , -0.3006029 ,  0.57529618],
             [-0.20486778, -0.42301292, -0.317077  ],
             [ 0.06988337, -0.17819288,  1.03969571],
             [-0.19603057, -0.42301292, -0.15714017],
             [-0.19122099, -0.3006029 , -0.41912632],
             [-0.19414242, -0.42301292, -0.12296806],
             [-0.17501962, -0.42301292,  0.22311842],
             [ 0.16985703, -0.17819288,  1.64280733],
             [ 1.12546259, -0.3006029 , 11.49561908]])
```

Clustering is distance-based and sensitive to feature magnitude. Without scaling, monetary features such as `total_spend` would dominate Euclidean distance, distorting cluster geometry. Standardisation ensures all features contribute equally.

```
[26]: #step 4

from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=5, random_state=42)
clusters = kmeans.fit_predict(X_scaled)

customer_df["Cluster"] = clusters

customer_df["Cluster"].value_counts()
```

```
[26]: Cluster
0      3769
4       499
3        33
2         7
1         4
Name: count, dtype: int64
```

```
[27]: customer_df.groupby("Cluster").mean()
```

```
[27]:
```

|         | Customer ID  | total_spend   | n_transactions | avg_basket_value |
|---------|--------------|---------------|----------------|------------------|
| Cluster |              |               |                |                  |
| 0       | 15413.051207 | 913.011326    | 3.197135       | 282.949553       |
| 1       | 15453.750000 | 236568.790000 | 118.500000     | 2444.268407      |

|   |              |              |           |             |
|---|--------------|--------------|-----------|-------------|
| 2 | 14875.142857 | 20889.344286 | 2.714286  | 8716.805286 |
| 3 | 15390.787879 | 38456.128848 | 62.121212 | 754.034533  |
| 4 | 14870.767535 | 6070.763054  | 9.258517  | 940.305514  |

```
[28]: from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=0.8, min_samples=10)
db_clusters = dbscan.fit_predict(X_scaled)

customer_df["DBSCAN_Cluster"] = db_clusters

customer_df["DBSCAN_Cluster"].value_counts()
```

```
[28]: DBSCAN_Cluster
0      4244
-1       68
Name: count, dtype: int64
```

Five clusters were selected as a balance between under-segmentation (3 clusters, which collapsed most customers into one group) and over-segmentation (10 clusters, which fragmented extreme customers into very small groups). The 5-cluster solution captures meaningful behavioural differences while maintaining interpretability.

Although the clusters are highly imbalanced, this reflects the skewed distribution of customer spending. A small number of customers contribute disproportionately high revenue, while the majority represent low to moderate purchasing behaviour.

## DBSCAN Analysis

DBSCAN identified one dominant dense cluster and 68 noise points. These noise points likely correspond to extreme high-spend customers. This suggests that the dataset is characterised by a large dense central customer base with a small number of behavioural outliers, rather than multiple well-separated dense segments.

## Conclusion

KMeans provides interpretable behavioural segmentation, while DBSCAN highlights the underlying density structure and the presence of extreme outliers. Together, the methods reveal a highly skewed customer distribution.

```
[29]: from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

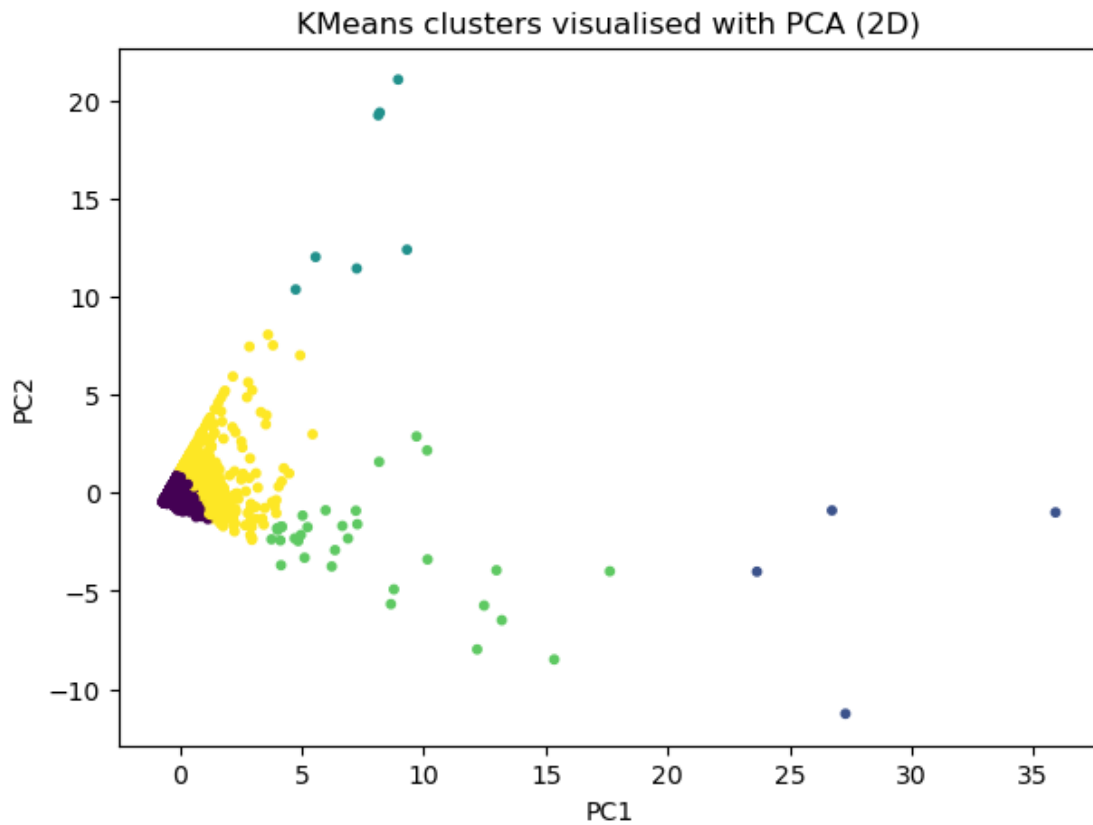
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(7,5))
plt.scatter(X_pca[:,0], X_pca[:,1], c=customer_df["Cluster"], s=10)
plt.title("KMeans clusters visualised with PCA (2D)")
```



```
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()

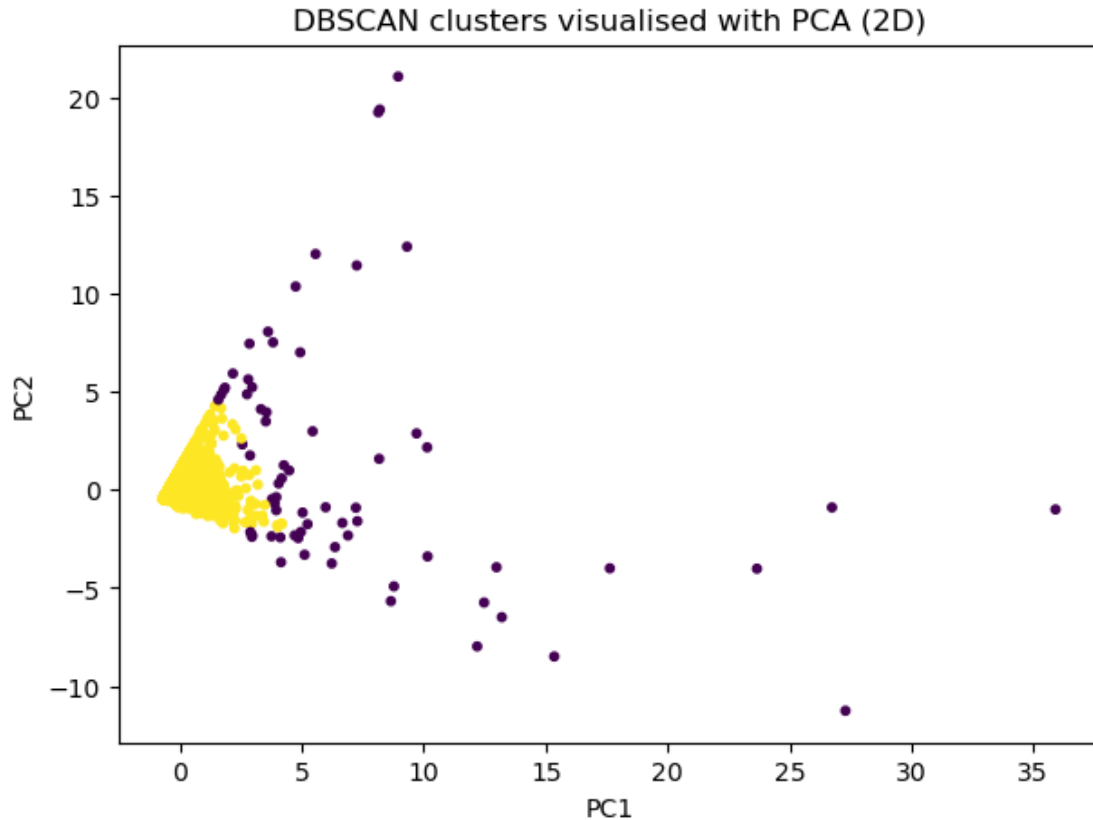
print("Explained variance ratio:", pca.explained_variance_ratio_)
```



Explained variance ratio: [0.58992251 0.309681 ]

```
[30]: plt.figure(figsize=(7,5))
plt.scatter(X_pca[:,0], X_pca[:,1],
            c=customer_df["DBSCAN_Cluster"],
            s=10)

plt.title("DBSCAN clusters visualised with PCA (2D)")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()
```



## Evaluation and Interpretation

### Interpretation of KMeans Clusters

The 5-cluster KMeans solution reveals meaningful behavioural differences between customers.

1. Cluster 0 (largest group): Customers in this cluster exhibit relatively low total spending, low transaction frequency, and moderate basket value. This represents the core mass of casual or low-to-moderate customers.
2. Cluster 4 (mid-sized group): These customers demonstrate higher total spend and higher transaction frequency compared to Cluster 0. They represent more engaged or loyal customers.
3. Cluster 3 (small group): Characterised by high transaction frequency and substantial total spend, suggesting frequent heavy purchasers.
4. Cluster 2 (very small group): These customers show high total spend but relatively low transaction frequency, combined with very high average basket value. This indicates bulk or high-value per-visit purchasers.
5. Cluster 1 (smallest group): Extreme high-spend, high-frequency customers. This cluster captures VIP or exceptional behavioural outliers.

Overall, the segmentation highlights strong heterogeneity in purchasing behaviour, particularly at the high end of spending.

### Cluster Balance

The clusters are highly imbalanced. The majority of customers fall into a single large cluster, while several clusters contain only a small number of customers.

However, this imbalance reflects the skewed nature of retail data, where a small number of customers often contribute disproportionately to total revenue. Therefore, the imbalance is not necessarily a modelling flaw but a structural characteristic of the dataset.

### DBSCAN Interpretation

DBSCAN identified one dominant dense cluster and 68 noise points. These noise points likely correspond to extreme high-spend customers.

This suggests that the dataset consists of a large dense central customer base with a small number of behavioural outliers, rather than multiple equally dense segments.

Compared to KMeans, DBSCAN does not force artificial partitions but instead highlights the natural density structure of the data.

### Limitations of the Approach

Several limitations should be acknowledged:

1. **Feature Selection:** Only three behavioural features were used (total spend, transaction frequency, and average basket value). Additional variables such as recency or product diversity could improve segmentation.
2. **Skewed Distribution:** The heavy-tailed spending distribution influences clustering results, particularly for distance-based methods like KMeans.
3. **Algorithm Assumptions:** KMeans assumes spherical clusters of similar variance, which may not fully reflect the underlying behavioural structure.
4. **Exploratory Nature:** This analysis is exploratory and does not validate clusters against external business outcomes.

### Sensitivity to Parameter Choices

The results are sensitive to parameter selection:

**KMeans:** Changing the number of clusters significantly alters segmentation. Using 3 clusters led to under-segmentation, while 10 clusters resulted in over-segmentation and fragmented micro-clusters. Five clusters provided a balanced and interpretable solution.

**DBSCAN:** Results depend on the choice of `eps` and `min_samples`. Different values may classify more or fewer points as noise.

This sensitivity highlights the importance of thoughtful parameter selection and interpretation rather than blindly optimising metrics.

[ ]: